SOLUTION BLUEPRINT

Hades Mirage

Layer-7 security for AI agents: catch rogue agents, divert them into an isolated honeypot, and explain every case in plain words with Google Gemini.

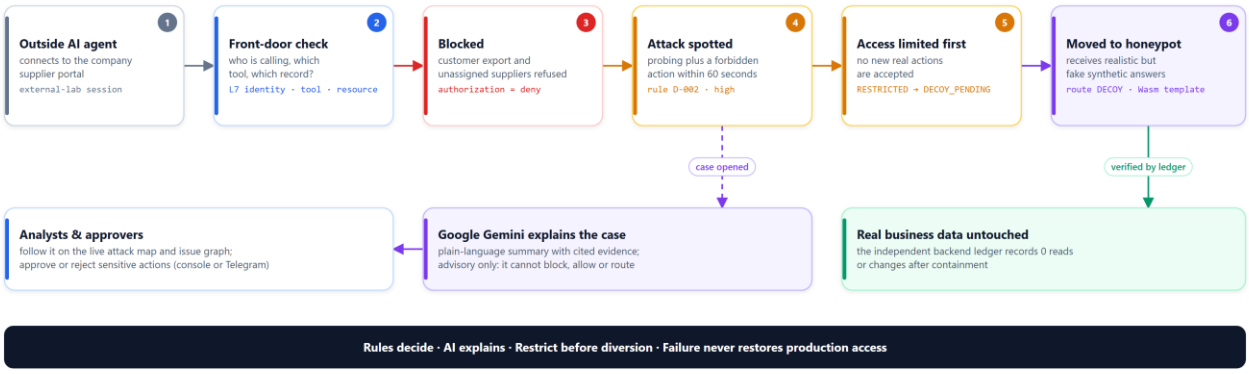


Figure 1. How Hades Mirage handles an attacking AI agent, from front door to honeypot.

Item	Detail
Version / date	1.0 · 18 October 2026
Status	Working prototype. P1-native profile verified on Windows without Docker; Google Cloud deployment scripted, not yet run
Google AI	Gemini via the Google Gen AI SDK, verified live with gemini-2.5-flash and gemini-3.8-flash
Data	All people, suppliers, records and attacks are synthetic laboratory fixtures

Contents

1. Executive summary 3
2. The problem 3
3. The solution at a glance 3
4. Where Hades Mirage sits 4
5. Solution architecture 5
   - Three planes 6
   - Technology 6
6. How an attack is handled 6
7. Detection rules 7
8. AI integration with Google Gemini 8
   - What Gemini does 8
   - Guardrails 8
   - Configured in the portal 9
   - Verified live results 9
9. Analyst experience 10
10. Approvals and Telegram 13
11. Google services and Cloud deployment 14
    - Deployment procedure 15
12. Security model and boundaries 15
13. Verification and results 16
14. Known gaps and roadmap 16

Appendix A. Run it locally (no Docker) 18

Appendix B. Key API endpoints 18

Appendix C. Selected configuration 18

1. Executive summary

Companies now let AI agents call internal tools, APIs and Model Context Protocol (MCP) servers. Network gateways decide which destinations an agent may reach, but once traffic is inside an encrypted tunnel they cannot tell a permitted supplier lookup from an unauthorised customer export or deletion.

Hades Mirage adds the missing application-layer control. It checks every request and tool call against the caller's identity, the tool, the exact resource and the action. Deterministic behaviour rules recognise probing and forbidden actions, restrict the session first and then quietly move eligible outside agents into an isolated synthetic honeypot, where they receive realistic but fake answers. An independent backend ledger proves whether real business data was touched.

Google Gemini turns the recorded evidence into plain-language investigations, answers analyst questions and reviews every new issue automatically. Gemini only advises: it has no tools, credentials or routing power, and every enforcement decision stays in deterministic code. Analysts work in a Wiz-style console with a live attack map, attack-path graphs that switch between plain and technical views, an AI chat assistant and approvals in the console or Telegram.

> **Measured in the local laboratory (P1-native, 18 October 2026)**
>
> 164 backend tests passed (6 explicit skips for unprovisioned gates) · opt-in live Gemini gate passed on two models with 0 backend reads or changes · a fresh external attack was diverted into the honeypot with 0 production reads or mutations and auto-reviewed by Gemini about 12 seconds later · the attack map reports 0 real-data changes after containment.

2. The problem

- **AI agents act with delegated power.** A prompt-injected or compromised agent can misuse tools it is legitimately allowed to reach.
- **Connectivity is not authorisation.** A certificate-bound HTTP/2 CONNECT gateway authorises destinations; it cannot see whether the encrypted operation inside the tunnel is allowed.
- **Blocking alone loses the story.** Hard blocks tip off attackers and end the investigation; analysts receive raw logs that are slow to interpret.
- **AI in the loop can become the risk.** If a model can approve, route or execute, prompt injection becomes an access-control bypass.

The goal is precise Layer-7 enforcement, behaviour-based detection, safe deception and explainable AI analysis, without ever giving the AI the keys.

3. The solution at a glance

Capability	What it does	How it is enforced
Layer-7 enforcement	Checks identity, tenant, tool, exact resource and action for every request and MCP tool call	Deny-by-default policy, explicit deny precedence, strict typed operation models

Capability	What it does	How it is enforced
Behaviour detection	Recognises corroborated probing, approval tampering, delegation escalation and injection with prohibited tools	Six deterministic rules (D-001 to D-006) over authenticated, tenant-bound events
Restrict, then divert	Stops new real actions first, then moves eligible outside agents into the honeypot	One-way route state machine with a durable dispatch fence
Synthetic honeypot	Returns realistic but fake supplier and invoice answers	WebAssembly module with no host imports, fuel and memory limits, synthetic state only
Independent evidence	Shows exactly what reached business systems	Signed loopback backend ledger, event digests, hash-chained audit log
AI investigation (Gemini)	Explains issues, answers questions, auto-reviews new issues, summarises reports	Advisory only: schema-validated, evidence-cited, budgeted, no tools or authority
Analyst console	Risk board, live attack map, issue drawer with attack graph, AI chat	Plain view for everyone, technical view for specialists; role-based access
Approvals	Sensitive actions need a separate approver, in the console or Telegram	Exact one-time digest approvals with expiry and replay protection
Deployment	Runs natively on Windows; packaged for Google Cloud	No Docker; Cloud Run source deploy with buildpacks, IAP, Vertex AI and Secret Manager

4. Where Hades Mirage sits

Hades Mirage operates primarily at OSI Layer 7, where it can understand users, agents, API requests, tool calls, resources and permitted actions. Layers 3 and 4 provide supporting isolation and connectivity controls.

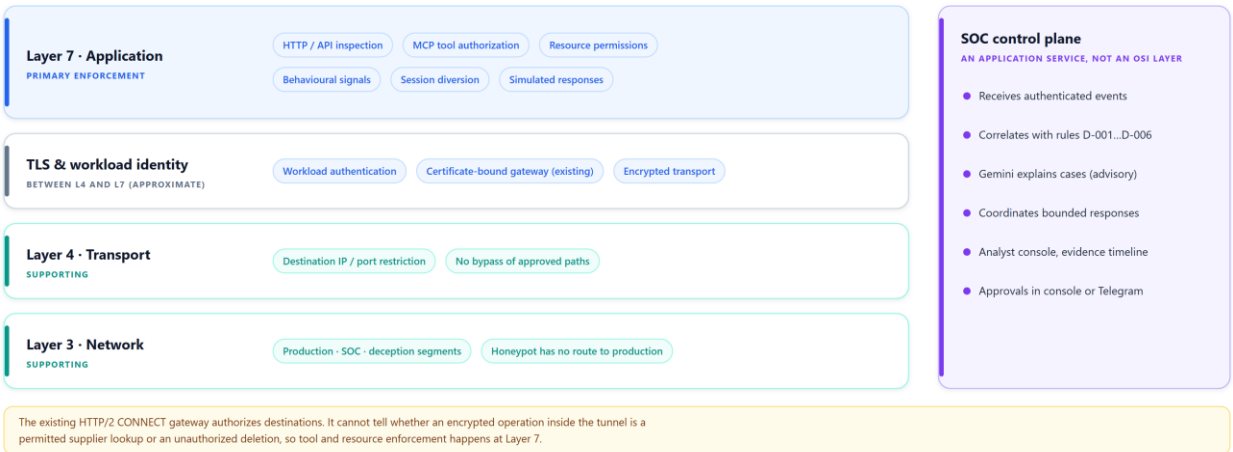


Figure 2. Control placement by layer. The SOC control plane is an application service, not a separate OSI layer.

OSI area	Where our controls fit
Layer 7 · Application (primary)	HTTP/API inspection, MCP tool authorisation, resource permissions, behavioural signals, session diversion and simulated application responses
Layer 3 · Network (supporting)	Segmentation and routing restrictions between production, SOC infrastructure and the deception environment
Layer 4 · Transport (supporting)	Restrict destination IP/port connectivity and prevent agents or decoys from bypassing approved paths
TLS and cryptographic identity	Authenticate workloads and protect communications; TLS sits between transport and application, so mapping it to one OSI layer is approximate
SOC control plane	Receives events, investigates cases and coordinates bounded responses; an application service rather than a separate OSI layer

5. Solution architecture

External users, bots and agents enter through the Layer-7 ingress, which authenticates, authorises and routes each request to the protected application, a rejection, or (for eligible restricted sessions) the isolated synthetic honeypot. Internal agents pass through Layer-7 tool and resource enforcement before the existing gateway checks identity and destination permissions. All components emit authenticated security events to the SOC control plane.

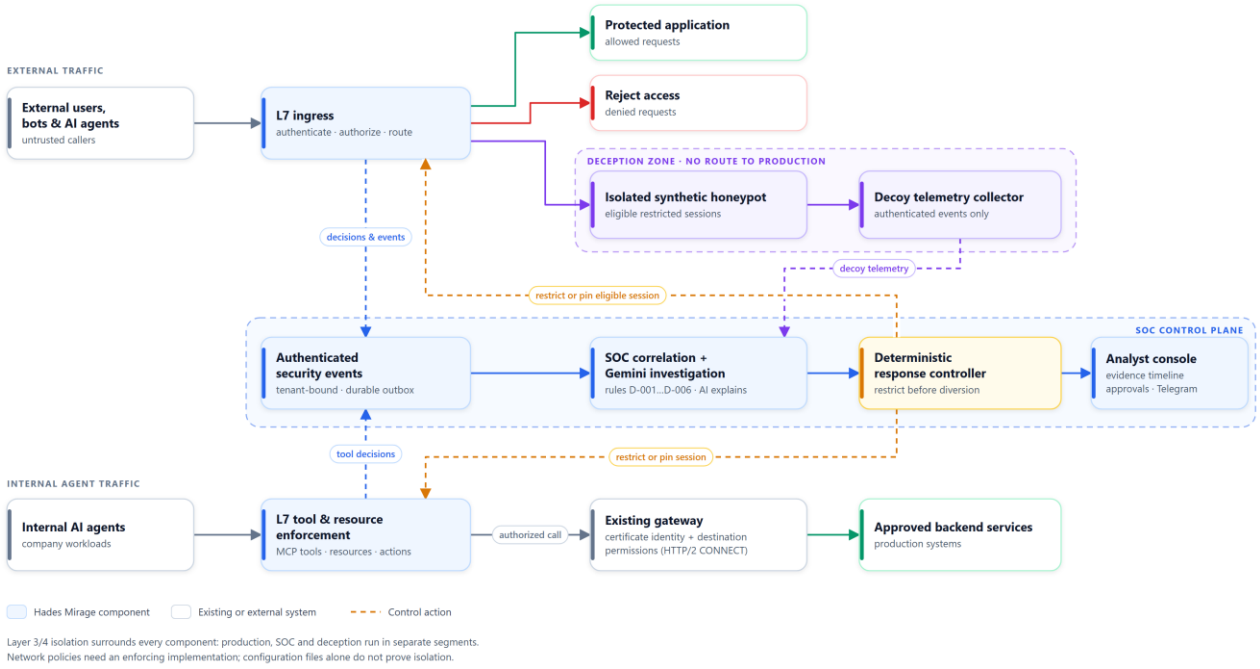


Figure 3. Solution architecture: enforcement, deception and SOC control planes with network isolation.

Three planes

Plane	Implemented components	Authority
Enforcement	Verified bearer/cookie context, strict operation models, deterministic policy, approvals and delegation, shared dispatch fence, signed HTTP backend	Decides and executes only authorised, typed operations
SOC control	Authenticated, tenant-bound events, transactional outbox and worker, rules D-001 to D-006, case and evidence store, notes and exports, Gemini interface	Deterministic response controller plus separately authenticated human roles
Deception	Trusted router and a reviewed no-import Wasmtime supplier module with per-session synthetic state	Bounded emulation only: no host callbacks, credentials, filesystem or network

Technology

- **Backend:** Python 3.13, FastAPI, Pydantic, SQLAlchemy (SQLite WAL; PostgreSQL path prepared), Wasmtime.
- **Console:** React and TypeScript with accessible Radix primitives, dark and light themes, plain and technical views.
- **AI:** Google Gen AI SDK (google-genai) calling Gemini through a server-side credential.
- **Operations:** native Windows launcher (setup.cmd, start.cmd) with no Docker; Google Cloud Run deployment script.

6. How an attack is handled

The console can tell every case in plain language for business readers or in exact technical terms for specialists. The table shows both views of the same recorded attack.

Step	Plain view	Technical view
1	An outside AI agent connects to the supplier portal	external-lab session created, route PRODUCTION
2	The front door checks who it is and what it wants	request.observed; identity, tool, resource and action evaluated
3	It tries to export customers and read unassigned suppliers: blocked	authorization.decided = deny (PROHIBITED_TOOL, RESOURCE_OUT_OF_SCOPE)
4	The pattern is recognised as an attack	Rule D-002 Corroborated resource probing, severity high; case opened
5	Its access is limited before anything else	Route RESTRICTED, then DECOY_PENDING after readiness checks

Step	Plain view	Technical view
6	It is moved into the honeypot and gets fake answers	Route DECOY; Wasm template responses; rule D-005 records decoy interaction
7	Real data is confirmed untouched	Independent ledger: 0 production reads or mutations after containment
8	Gemini explains the case; analysts review and decide	Advisory investigation with cited event IDs; analyst notes and actions audited

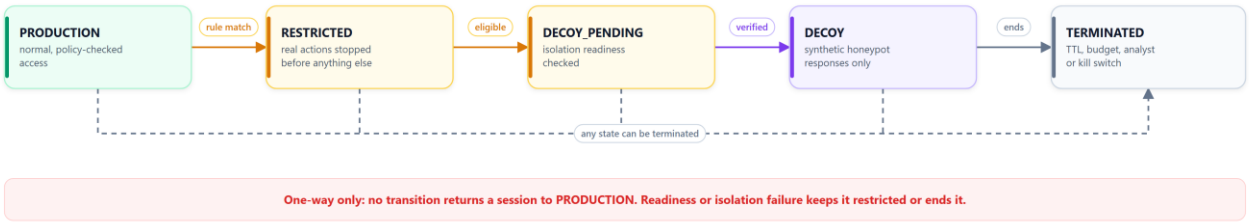


Figure 4. Session route states. Transitions are one-way; nothing returns a session to PRODUCTION.

Design rule: restrict before diversion

A session is restricted before it can be diverted. If the honeypot is not verified ready, or isolation fails, the session stays restricted or is terminated. Failure never restores production access.

7. Detection rules

Detection is deterministic and versioned (rules-1.0.0). A rule match is a reason to investigate, not proof of malicious intent.

Rule	Name	Threshold	Response	Severity
D-001	Distinct-resource access denials	5 distinct resources in 60 s	Restrict	Medium
D-002	Corroborated resource probing	Prohibited tool + 3 denied lookups in 60 s	Eligible deception	High
D-003	Approval verification failure	1 invalid action-bound approval	Review	Medium
D-004	Delegation escalation	1 attempt exceeding ancestor authority	Review	Medium
D-005	Verified decoy interaction	1 observation from the isolated template	Observe	Medium

Rule	Name	Threshold	Response	Severity
D-006	Instruction override with prohibited tool	Injection-like text + prohibited action in 60 s	Eligible deception	High

8. AI integration with Google Gemini

Gemini runs on a separate advisory path. Evidence flows one way from the enforcement path into a bounded context; Gemini's structured answer is validated before it is shown. Nothing the model writes can change permissions, routing or approvals.

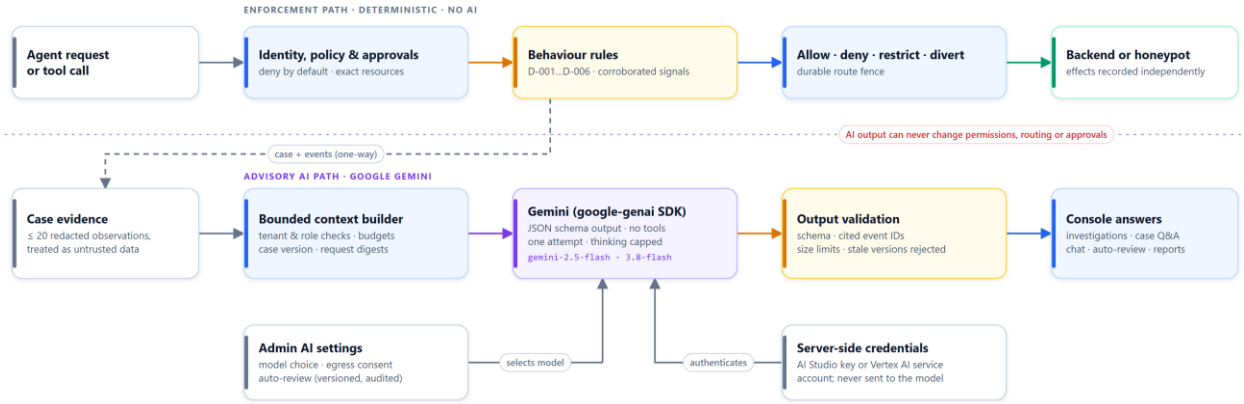


Figure 5. Deterministic enforcement path and the advisory Gemini path.

What Gemini does

- **Automatic issue review:** when enabled by an administrator, every new issue receives a bounded investigation.
- **Investigations:** structured explanations with confidence, alternative explanations and evidence gaps, each claim citing event IDs.
- **Case questions:** read-only answers to analyst questions about one case, for example "In plain words, what happened?"
- **Ask Mirage AI:** a workspace chat that answers across issues with validated references to real cases.
- **Report summaries:** metadata-only summaries of imported Hades scanner reports (no raw descriptions or source code).
- **Decoy wording (optional):** display-only wording around an unchanged deterministic honeypot response.

Guardrails

- No tools: tools are empty and automatic function calling is disabled; any tool call in the output is rejected.
- Structured output: every request uses a JSON schema; invalid, truncated or oversized answers are rejected.
- Evidence discipline: at most 20 redacted case observations, treated as untrusted data; cited event IDs and case versions are verified.

- Budgets and idempotency: tenant and per-case call budgets, pending limits, input/output bounds and one attempt per request.
- Credentials stay server-side: a Gemini API key in the ignored .env or Secret Manager, or the Cloud Run service account on Vertex AI; never in the browser or the prompt.
- Hidden reasoning is capped per model family, so thinking tokens cannot truncate the visible answer.
- Clear failures: billing, key, model, rate-limit, unsupported-request and timeout errors are reported in plain words; deterministic controls continue.

Configured in the portal

Administrators choose Demo mode, Google Gemini or AI off; pick the model from the live list available to the key; acknowledge that synthetic case evidence may be sent to Gemini; and switch automatic review on or off. Settings are versioned, audited and apply without a restart.

Verified live results

Opt-in five-call live gate over the real signed backend HTTP path (18 October 2026):

Workflow	gemini-2.5-flash	gemini-3.8-flash
Investigation	5.6 s	8.3 s
Case question	5.5 s	7.8 s
Provider check	2.7 s	3.6 s
Report summary	5.0 s	6.4 s
Decoy wording	2.8 s	3.5 s
Backend reads / mutations caused	0 / 0	0 / 0

In the running console with gemini-2.5-flash: the configuration check passed in 3.1 s, Ask Mirage AI answered with 7 cited references in 4.5 s, a new honeypot issue was reviewed automatically about 12 s after creation, and a plain-language case question was answered in 7.9 s. Accuracy and latency under load were not measured.

9. Analyst experience

The console follows the patterns of modern cloud-security platforms: a risk board first, issues with a side drawer, an interactive attack graph and an always-available AI assistant. A global switch moves every screen between plain and technical language.

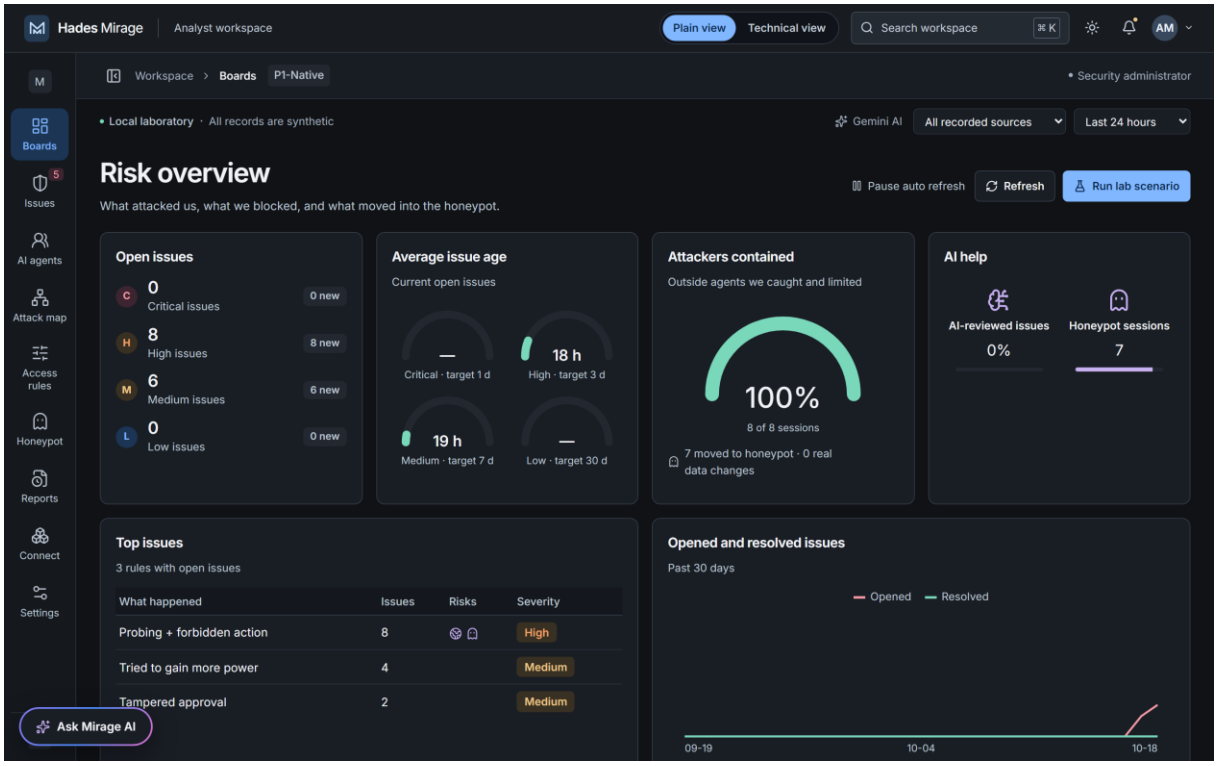


Figure 6. Risk overview: open issues, issue age, containment and AI help at a glance.

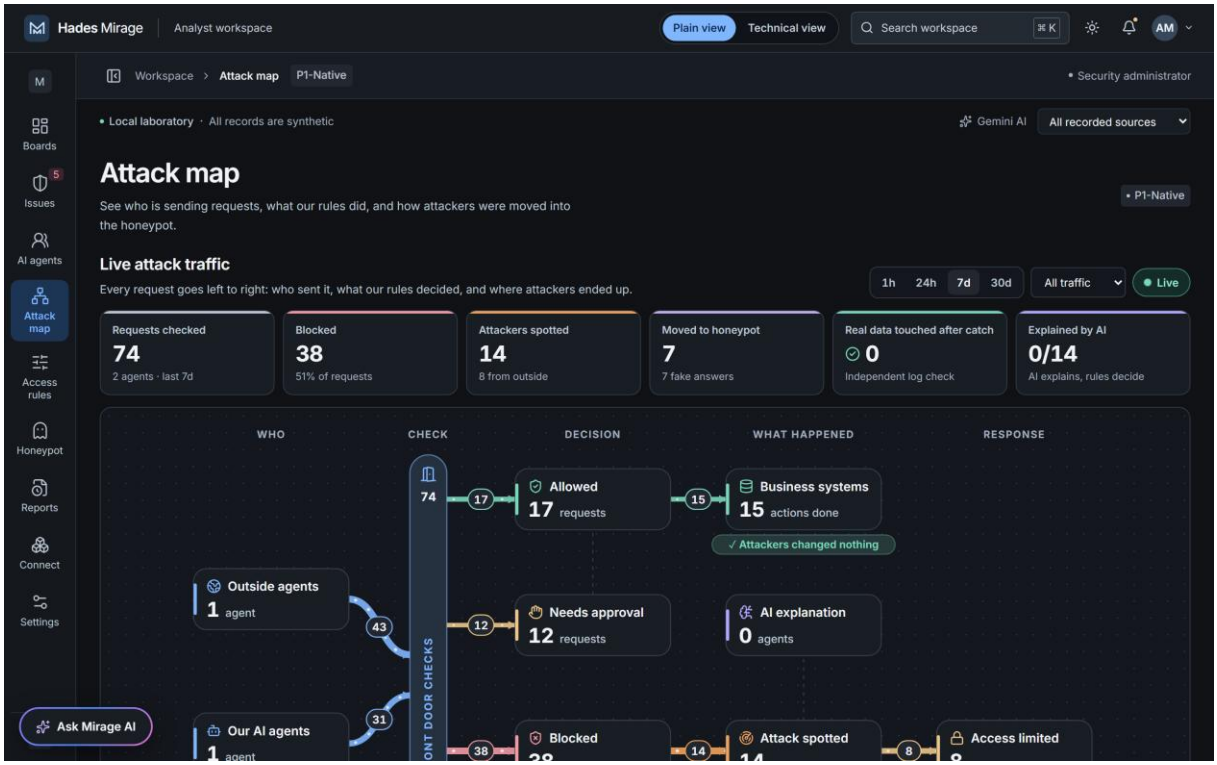


Figure 7. Live attack map: who sent requests, what the rules decided and where attackers ended up.

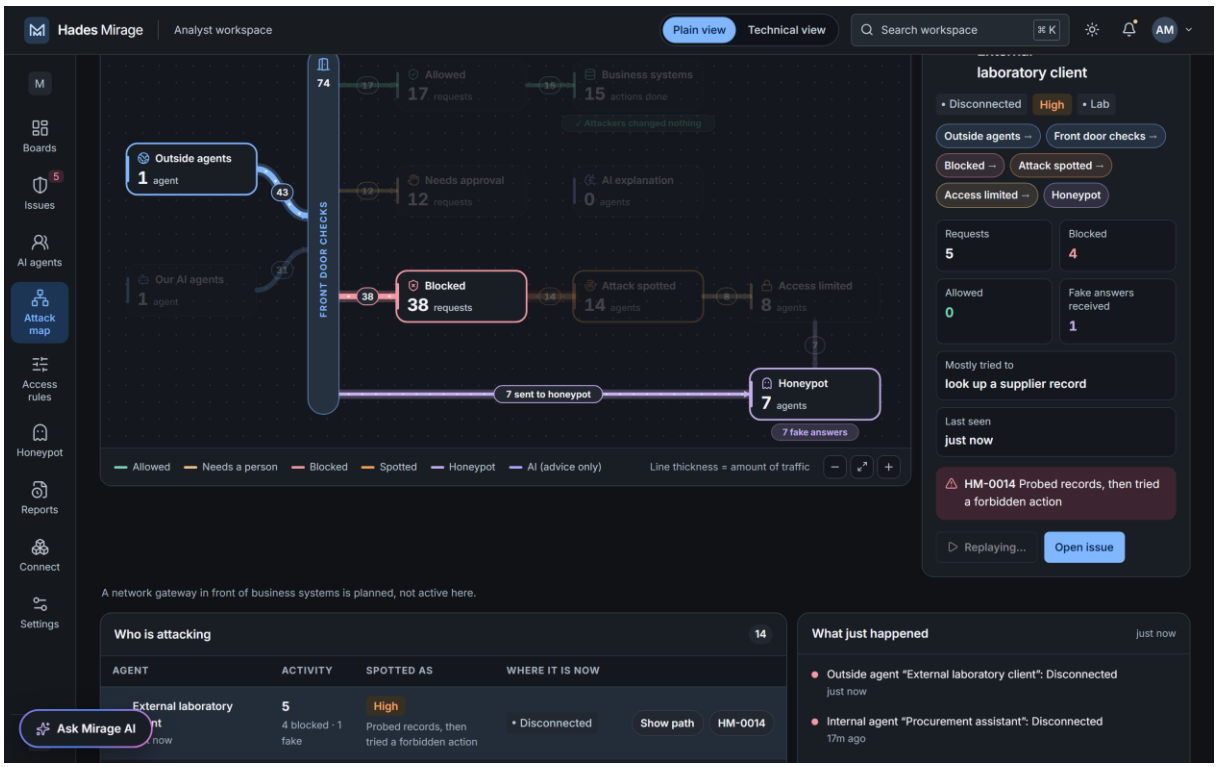


Figure 8. Attack path replay for one outside agent, with its session details and linked issue.

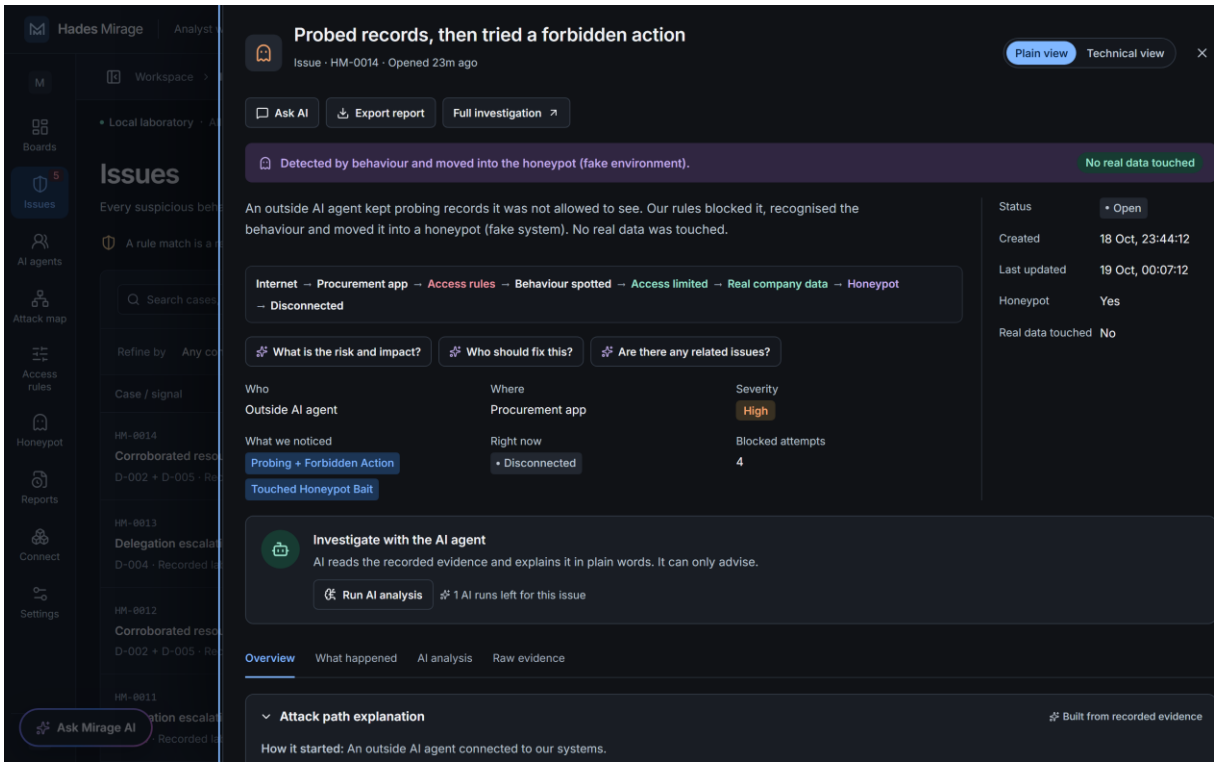


Figure 9. Issue drawer in plain view: what happened, where, how severe, and that no real data was touched.

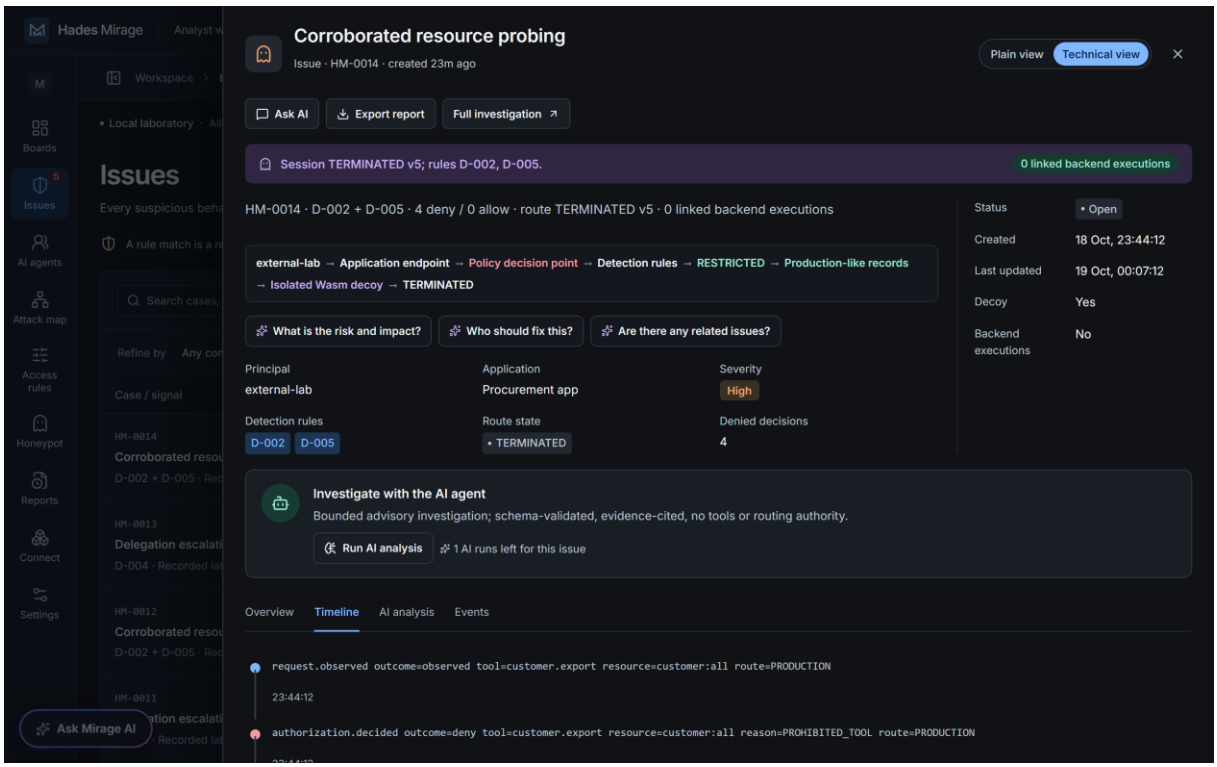


Figure 10. The same issue in technical view: rules, route state, denied decisions and raw event timeline.

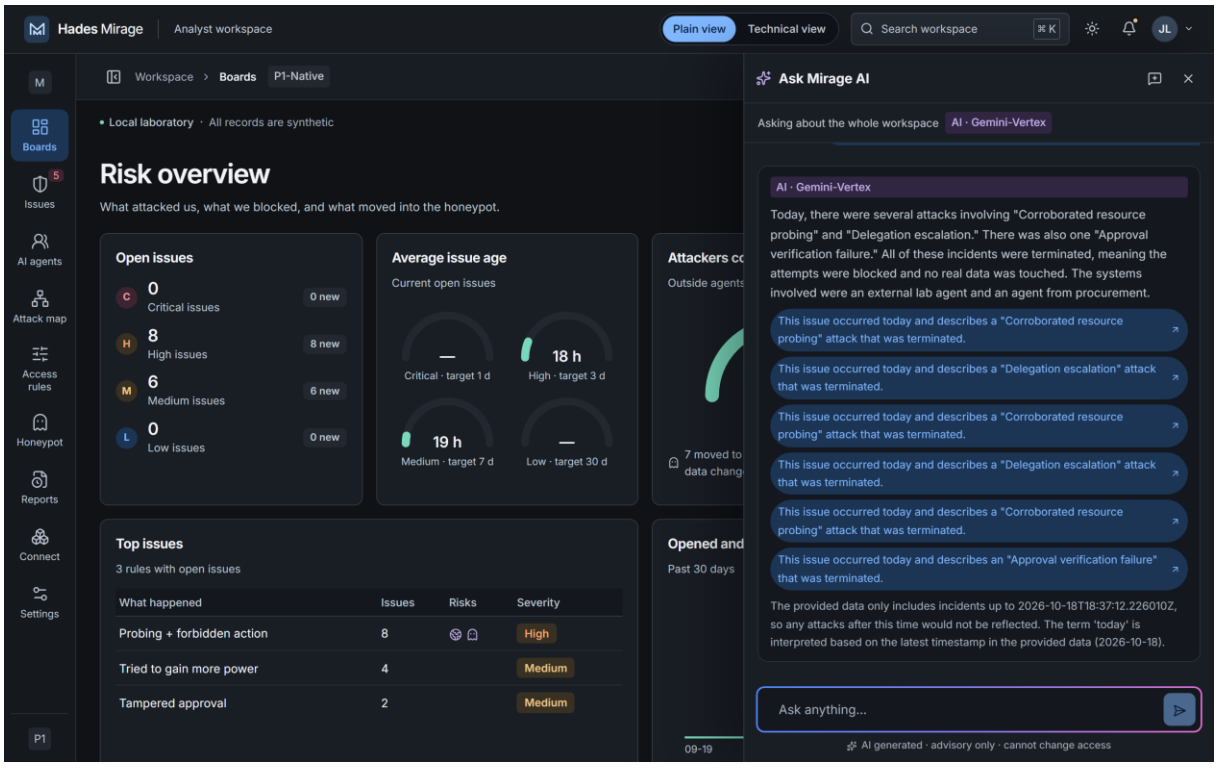


Figure 11. Ask Mirage AI answering with live Gemini and links to the issues it cites.

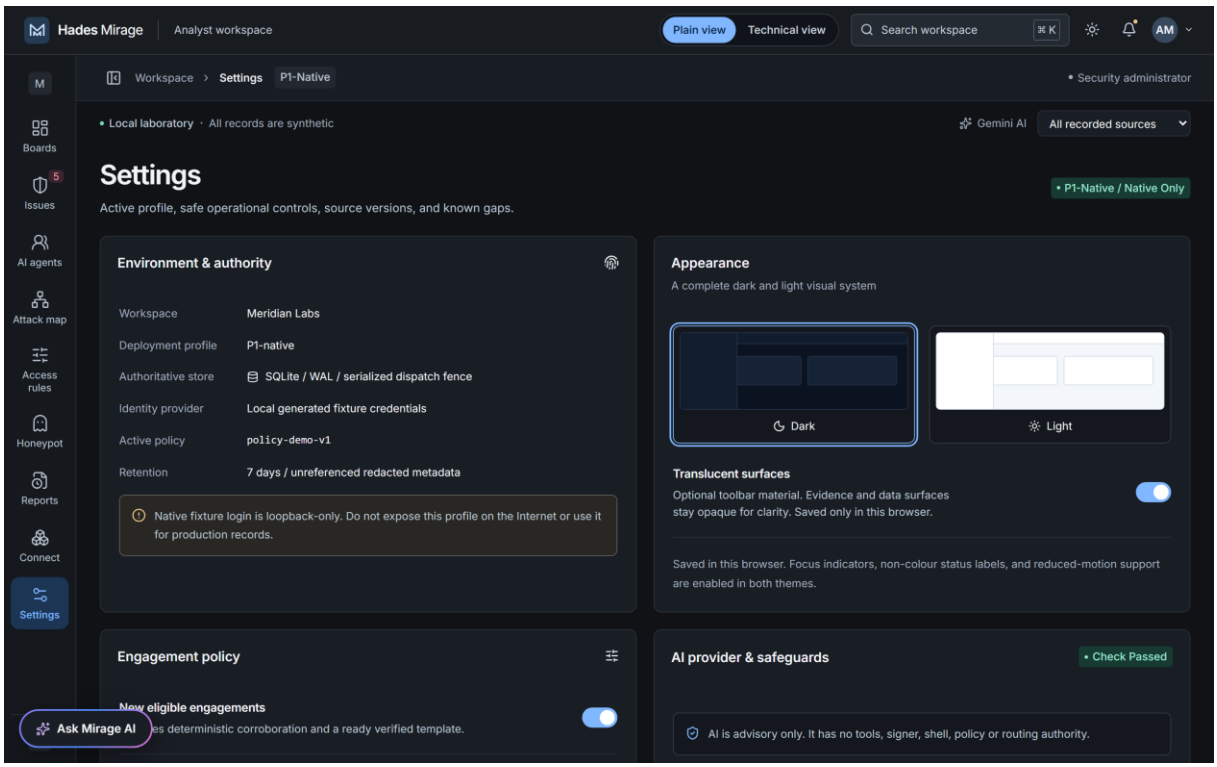


Figure 12. Settings: AI provider and safeguards, with the live Gemini check passed.

10. Approvals and Telegram

- Sensitive operations such as payment changes require an exact, one-time approval. The approval digest binds the action, arguments, session and active policy; the requester cannot approve their own request.
- Telegram users request access through the bot. An administrator approves or rejects each person and assigns a viewer, analyst or approver role; Telegram can never grant administrator rights.
- Approvers receive role-limited alerts with Approve and Reject buttons that reuse the console checks: exact digest, separate reviewer, expiry and replay protection.
- Status: tested end to end against a fake Telegram API; live delivery from this laboratory network is blocked by the corporate web gateway.

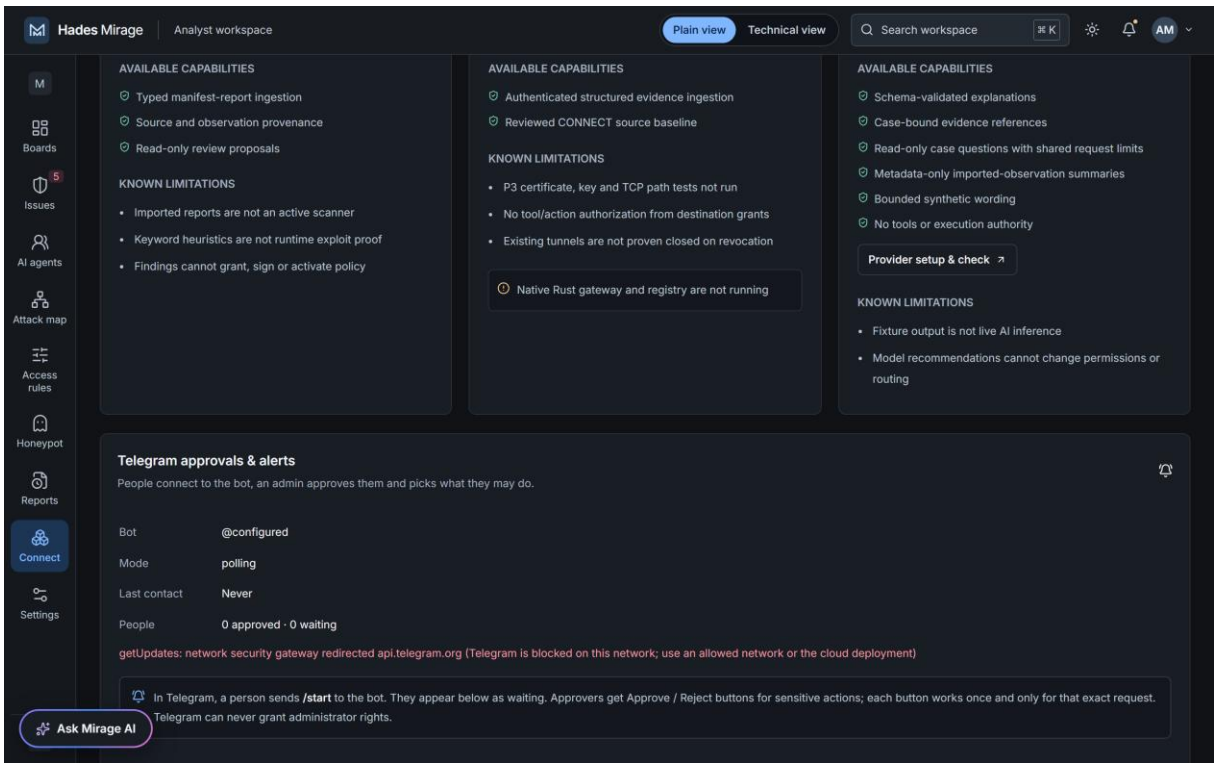


Figure 13. Connect: Telegram approvals and alerts managed by an administrator.

11. Google services and Cloud deployment

Google service	Role in Hades Mirage	Status
Gemini API (Google AI Studio) via Google Gen AI SDK	Investigations, case questions, workspace chat, automatic issue review, report summaries, decoy wording	**Verified live**
Vertex AI · Gemini	Cloud inference with the Cloud Run service account (roles/aiplatform.user); no API keys in the cloud	Implemented; not exercised live
Cloud Run	Hosts the API and console (profile P2-cloud) with the signed synthetic backend on container loopback	Scripted; not deployed
Cloud Build + Google Cloud buildpacks	Builds the image from source with no Dockerfile and no local Docker	Scripted
Artifact Registry	Stores the built image (cloud-run-source-deploy)	Scripted
Identity-Aware Proxy + Google Workspace	Workspace sign-in; the app verifies the signed IAP JWT (ES256, issuer, audience, expiry, domain)	Tested locally with generated keys
Secret Manager	Backend HMAC secret and Telegram bot token	Scripted
Cloud Logging	Collects stdout and stderr from both processes	Scripted

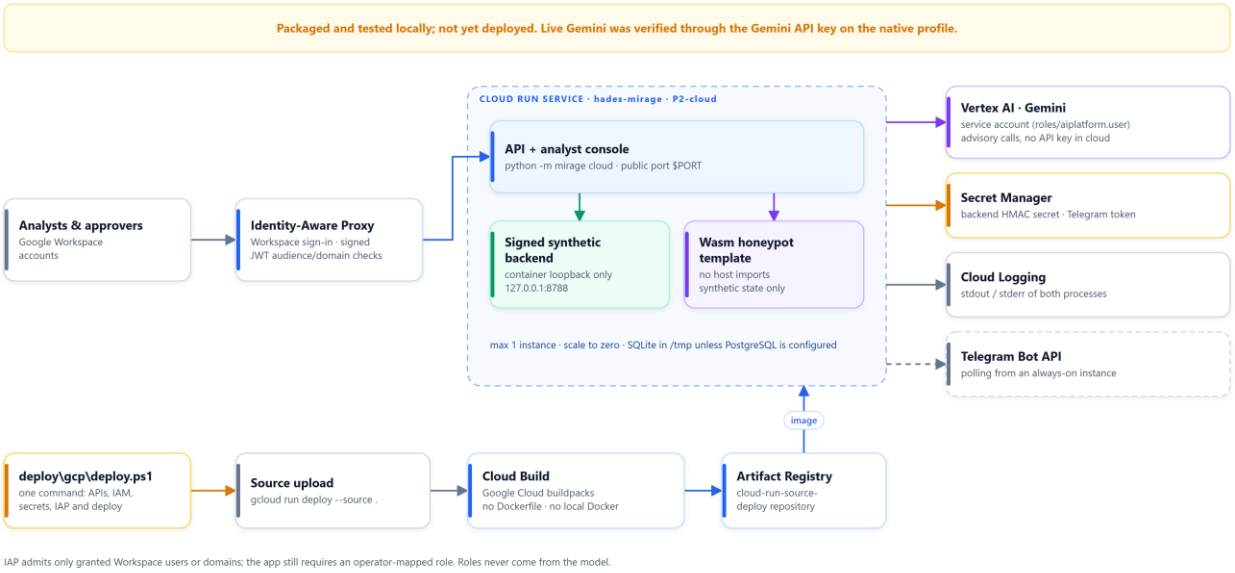


Figure 14. Google Cloud deployment blueprint: Cloud Run behind IAP with Vertex AI, Secret Manager and Cloud Logging.

Deployment procedure

1. Prepare a Google Cloud project with billing inside the Google Workspace organisation, and an account with Cloud Run, IAP, IAM, Secret Manager, Service Usage and Cloud Build roles.
2. Run deploy\gcp\deploy.ps1 with the project, region, allowed Workspace domain and role map.
3. The script enables the required APIs, creates a least-privilege service account, generates and stores secrets, and runs gcloud run deploy --source . with --iap, one maximum instance and scale to zero.
4. It then grants the IAP service agent permission to invoke the service and gives mapped users IAP access. Roles inside the app come from operator configuration.

> **Keep the existing gateway's client certificates on a compatible TLS endpoint**
>
> Cloud Run terminates inbound TLS before forwarding requests to the container. The existing gateway's native client-certificate verification therefore needs its own TLS endpoint; Cloud Run hosts the application and SOC services.

12. Security model and boundaries

- Enforcement lives outside the model. Gemini never receives a signer, router, shell or backend credential.
- Tenant, identity, resource, action and approval checks run on the server for every request.
- Prompts, tool output, scanner text, event snippets and model answers are treated as untrusted data; the console never renders attacker or model HTML.
- Decoys are synthetic and isolated with no production or external connectivity; attackers never execute arbitrary commands.
- Restrict before diversion; failures keep sessions restricted or end them, never restoring production access.
- Destination (TCP) authorisation is not treated as tool authorisation.
- Evidence carries server receipt times, body digests and a hash-chained audit log; application integrity, not externally anchored storage.

- Secrets stay server-side (ignored .env locally, Secret Manager in the cloud) and are never returned, logged or placed in prompts.

13. Verification and results

Check	Result
Backend test suite (pytest)	164 passed, 6 explicit skips (P3 gateway ×3, PostgreSQL, cloud deployment, live gate in the default run)
Live Gemini gate (opt-in, five real calls)	Passed on gemini-2.5-flash and gemini-3.8-flash; 0 backend reads/mutations; case route unchanged
Browser workflows (Playwright)	29 of 29 passed in the last full run, including attack-map accessibility scans in both themes
Static checks	Ruff, ESLint, Prettier, TypeScript and the production build passed
Blueprint validator	200 documentation and fixture checks passed; OpenAPI contract unchanged
Secret scan	The Gemini key and Telegram token appear only in the ignored local .env
Running demo (P1-native)	Fresh external attack diverted to the honeypot with 0 production reads/mutations; Gemini auto-review about 12 s later

> **Not claimed**
>
> Model accuracy or detection quality, latency under load, Vertex AI workload identity, any Cloud Run deployment, live Telegram delivery, the P3 native gateway and PostgreSQL operation. Laboratory results use synthetic data.

14. Known gaps and roadmap

Gap	Why it remains	Next step
P3 native gateway (certificates, OS egress fencing, tunnel revocation)	No Rust toolchain, test PKI or signed registry provisioned	Provision and run the gateway gates T-08, T-09 and T-25
Vertex AI workload identity	No authorised Google Cloud project	Deploy, then run the live gate on Vertex AI
Cloud Run deployment	No cloud credentials or spend authorised	Run deploy.ps1; validate IAP, origin and IAM paths

Gap	Why it remains	Next step
Live Telegram delivery	Corporate web gateway blocks the Bot API	Use an allowed network or the Cloud Run instance
PostgreSQL	Code path prepared but not run against a server	Run the acceptance suite on PostgreSQL
Enterprise identity	Local fixture identities stand in for SSO and workload identities	Integrate approved identity providers
Independent review	Security, accessibility, load and recovery not externally reviewed	Schedule reviews before any pilot

Appendix A. Run it locally (no Docker)

1. Run .\setup.cmd once: it creates the Python environment from the locked requirements, builds the console and prepares the database.
2. Run .\start.cmd (or .\.venv\Scripts\python.exe -m mirage run) and open http://127.0.0.1:8787.
3. Sign in as admin, analyst or approver with the local access code printed by the launcher (also stored in .local\runtime-secrets.json).
4. For live AI, put MIRAGE_GEMINI_API_KEY in .env, then Settings → AI provider → Google Gemini → choose a Flash model → acknowledge → Save → Check configured AI.
5. On Boards, choose Run lab scenario, then follow the attack on the Attack map and in Issues.

Appendix B. Key API endpoints

Endpoint	Purpose
GET /health	Readiness and active profile
GET /v1/board · GET /v1/overview	Risk board and overview metrics
GET /v1/attack-map	Live attack map (1 h, 24 h, 7 d, 30 d windows)
GET /v1/incidents · GET /v1/incidents/{id}	Issues and case detail with evidence
GET /v1/incidents/{id}/attack-path	Plain and technical attack story
POST /v1/incidents/{id}/investigate	Gemini investigation (advisory)
POST /v1/incidents/{id}/assistant · POST /v1/assistant	Case questions and workspace chat
GET /v1/ai · PUT /v1/ai/config · GET /v1/ai/models · POST /v1/ai/check	AI status, portal configuration, model list and live check
GET /v1/deception · POST /v1/deception/verify · POST /v1/deception/kill-switch	Honeypot readiness, isolation probes and kill switch
POST /v1/laboratory/run	Run a bounded synthetic attack scenario

Appendix C. Selected configuration

Variable	Meaning
MIRAGE_PROFILE	P1-native (local) or P2-cloud (Cloud Run)
MIRAGE_MODEL_PROVIDER	fixture (Demo mode), gemini, or unavailable (AI off)
MIRAGE_GEMINI_API_KEY / MIRAGE_GEMINI_ENDPOINT	Server-side Gemini API key and endpoint selection (auto, developer or vertex)

Variable	Meaning
MIRAGE_GEMINI_MODEL / PROJECT / LOCATION	Default model; project and region for Vertex AI
MIRAGE_MODEL_EGRESS_APPROVED	Operator approval to send synthetic evidence to Gemini
MIRAGE_MODEL_CALL_BUDGET / MIRAGE_MODEL_CASE_CALL_BUDGET	Tenant and per-case AI call budgets
MIRAGE_TELEGRAM_MODE / MIRAGE_TELEGRAM_BOT_TOKEN	Telegram polling or webhook mode and bot token
MIRAGE_SESSION_TTL_SECONDS / MIRAGE_DECOY_REQUEST_BUDGET / MIRAGE_MAX_ENGAGEMENTS	Honeypot engagement limits